

Klasyfikacja fake-news: sieć neuronowa TF-IDF → MLP

Dokumentacja techniczna, Sztuczne Sieci Neuronowe

Aleksander Oleszkiewicz

24 czerwca 2026

Streszczenie

Dokument stanowi pełną dokumentację techniczną końcowego modelu projektu, czyli binarnego klasyfikatora fałszywych wiadomości opartego na reprezentacji **TF-IDF** i wielowarstwowej sieci gęstej (**MLP**) zbudowanej od zera w PyTorchu. Omówiono analizę i czyszczenie danych (w tym eliminację wycieku informacji), architekturę i proces uczenia modelu, metryki ewaluacji wraz z diagnozą bias-variance, kluczowe komponenty kodu oraz przykładowe predykcje. Model osiąga F1 i ROC-AUC na poziomie $\sim 0,99$ na zbiorze testowym.

Spis treści

1	Wstęp	2
1.1	Problem	2
1.2	Cel	2
1.3	Zbiór danych	2
2	Analiza i przygotowanie danych	2
2.1	Statystyki zbioru	2
2.2	Wyciek informacji (data leakage)	3
2.3	Czyszczenie tekstu	3
2.4	Podział na zbiory	3
3	Architektura modelu	4
3.1	Reprezentacja tekstu: TF-IDF	4
3.2	Sieć gęsta (MLP)	4
3.3	Proces uczenia	4
4	Ewaluacja	5
4.1	Metryki na walidacji i teście	5
4.2	Porównanie z baselinem (LogReg vs MLP)	6
4.3	Diagnoza bias-variance	7
4.4	Techniki regularyzacji	7
4.5	Interpretowalność cech	8
5	Przykładowe predykcje	9
6	Kluczowe komponenty kodu	10
6.1	TfidfDataset: leniwa densyfikacja	10
6.2	MLPClassifier: sieć zwracająca logity	11
6.3	train_mlp: pętla treningowa z early stopping	11

6.4	<code>predict_proba</code> i <code>evaluate_mlp</code>	12
6.5	Reprodukowalność i urządzenie	12
7	Wnioski i dalsze kroki	12
7.1	Podsumowanie	12
7.2	Ograniczenia	12
7.3	Dalsze kroki	12

1 Wstęp

1.1 Problem

Automatyczne wykrywanie fałszywych wiadomości (*fake-news*) to zadanie klasyfikacji binarnej: na podstawie treści artykułu prasowego model rozstrzyga, czy pochodzi on z wiarygodnego źródła (*Real*), czy jest dezinformacją (*Fake*). Trudność problemu nie leży w samym dopasowaniu klasyfikatora (jak pokazano dalej, klasy są w tym zbiorze silnie rozróżnialne leksykalnie), lecz w **rzetelnym przygotowaniu danych**: zbiór zawiera liczne sygnały wycieku informacji (ang. *data leakage*), które pozwalają uzyskać niemal perfekcyjne metryki bez uczenia się treści. Większość pracy projektowej dotyczy więc identyfikacji i usunięcia tych sygnałów, tak by model uczył się stylu i tematyki, a nie artefaktów technicznych.

1.2 Cel

Celem projektu jest zbudowanie **sieci neuronowej od zera** (bez wag pretrenowanych, bez fine-tuningu i feature-tuningu, zgodnie z wymogiem przedmiotu SSN) oraz zweryfikowanie kompletnej pętli treningowej w PyTorchu (propagacja wsteczna, optymalizator Adam, early stopping, regularyzacja, obsługa GPU). Kończącym modelem projektu, opisanym w niniejszej dokumentacji, jest klasyfikator **TF-IDF** → **MLP**.

1.3 Zbiór danych

Wykorzystano publiczny zbiór *Fake and Real News Dataset* (Kaggle, [clmentbisailon/fake-and-real-news-da](https://www.kaggle.com/clmentbisailon/fake-and-real-news-dataset)) czyli dwa pliki CSV (`True.csv`, `Fake.csv`) z artykułami prasowymi w języku angielskim. W całym projekcie obowiązuje konwencja etykiet:

1 = Real, 0 = Fake.

Model operuje wyłącznie na kolumnie `text`; kolumny `title`, `subject` i `date` są odrzucane jako źródła wycieku informacji (uzasadnienie w sekcji 2).

2 Analiza i przygotowanie danych

2.1 Statystyki zbioru

Surowy zbiór liczy **44 898** artykułów o niemal zrównoważonych klasach (~52% Fake / 48% Real). Eksploracyjna analiza danych (`notebooks/explore/01_eda.ipynb`) ujawniła dwa istotne problemy jakości:

- **Duplikaty**: ok. 13,9% artykułów to dokładne duplikaty tekstu, skupione niemal wyłącznie w klasie Fake (ok. 25,7% artykułów Fake vs 1,1% Real). Brak konfliktów etykiet między klasami, zatem deduplikacja jest bezpieczna.
- **Puste / bardzo krótkie teksty**: ok. 1,4% rekordów (głównie wpisy Fake zawierające jedynie odnośnik do YouTube).

Po pełnym potoku czyszczenia i deduplikacji pozostaje **38 470** artykułów (Fake: 17 281 = 44,9%, Real: 21 189 = 55,1%). Rozkład długości tekstu jest silnie prawoskośny; 95. percentyl to ok. **900 słów**, co bywa istotne przy ewentualnym rozszerzeniu o model sekwencyjny (kandydat na `max_seq_len`).

2.2 Wyciek informacji (data leakage)

Analiza w `notebooks/explore/02_leakage.ipynb` wykorzystwała informację wzajemną (Mutual Information, MI) oraz test χ^2 do oceny siły poszczególnych markerów jako predyktorów klasy. Najsilniejsze sygnały (Tabela 1) to artefakty redakcyjno-techniczne, a nie cechy treści: nagłówki agencyjny “(Reuters)” pozwala niemal jednoznacznie wskazać klasę Real.

Tabela 1: Najsilniejsze markery wycieku wg informacji wzajemnej (MI).

Marker	MI [nat]	Dominująca klasa
słowo „reuters”	0,650	Real
dateline „MIASTO (Reuters) – ”	0,456	Real
„featured image”	0,136	Fake
@handle (media społ.)	0,076	Fake
„getty (images)”	0,061	Fake
prefiks clickbait (VIDEO/WATCH)	0,018	Fake

Wniosek z analizy: kolumna `subject` rozdziela klasy w 100% (każda kategoria należy w całości do jednej klasy), a `title` oraz `date` również niosą silny wyciek (Fake: 83% tytułów ALL-CAPS; daty Fake w niestandardowym formacie). Wszystkie trzy kolumny są więc odrzucane, a model korzysta wyłącznie z `text`.

2.3 Czyszczenie tekstu

Funkcja `clean_text` (`src/data/cleaning.py`) stosuje uporządkowaną kaskadę wyrażeń regularnych, w której każdy wzorzec ma uzasadnienie w analizie MI powyżej. Kolejność operacji jest istotna (np. usunięcie „via @handle” przed samym @handle, rozdzielenie „2017World” przed sprowadzeniem do małych liter):

1. markery agencyjne Reuters (dateline, „(Reuters)”, słowo „reuters”);
2. adresy URL (`http(s)`, `www`, `pic.twitter.com`, `tmsnrt.rs`);
3. uchwyt mediów społecznościowych (@handle, „via @...”);
4. prefiksy clickbait (VIDEO:, WATCH:, BREAKING:);
5. szablony podpisów zdjęć („featured image via getty”, „photo by...”);
6. artefakty HTML (znaczniki, encje, bloki `<script>`);
7. sygnatury serwisów („21st Century Wire”);
8. normalizacja: białe znaki, usunięcie pojedynczych znaków, małe litery.

Po czyszczeniu stosowany jest filtr `filter_short_articles` z progiem `min_words = 10` (usuwa głównie wpisy-zaślepki Fake). Walidacja w `04_post_cleaning.ipynb` potwierdza, że kluczowe markery (dateline, URL, @handle, getty) spadają do 0% wystąpień, a pokrycie słownika GloVe pozostaje wysokie ($OOV \leq 0,2\%$).

2.4 Podział na zbiory

Funkcja `make_splits` (`src/data/splits.py`) dzieli dane **stratyfikowanie** po etykiecie na zbiory `train/val/test` w proporcji 60/20/20 (dwukrotne wywołanie `train_test_split`), ze stałym

ziarnem `RANDOM_STATE` dla powtarzalności (Tabela 2). Balans klas ($\sim 44,9\%$ Fake / $55,1\%$ Real) jest zachowany we wszystkich trzech zbiorach.

Tabela 2: Podział stratyfikowany na zbiory.

Zbiór	Liczba artykułów	Udział
train	23 082	60%
val	7 694	20%
test	7 694	20%
razem	38 470	100%

3 Architektura modelu

3.1 Reprezentacja tekstu: TF-IDF

Tekst jest zamieniany na wektor cech metodą **TF-IDF** (*Term Frequency-Inverse Document Frequency*). Jest to nieparametryczna inżynieria cech (*nie* model pretrenowany), więc model neuronowy w całości uczy się od zera. Wektoryzator (`make_tfidf_vectorizer`, `src/model/baseline.py`) jest **współdzielony** z baselinem (regresją logistyczną), co zapewnia uczciwe porównanie obu klasyfikatorów. Jest dopasowywany **wyłącznie na zbiorze treningowym**, a tym samym słownikiem transformowane są val i test (brak wycieku ze zbiorów ewaluacyjnych). Parametry zebrano w Tabeli 3.

Tabela 3: Konfiguracja wektoryzatora TF-IDF.

Parametr	Wartość
<code>max_features</code>	50 000 (rozmiar słownika)
<code>ngram_range</code>	(1, 2): unigramy i bigramy
<code>min_df</code>	5 (minimalna częstość dokumentowa)
<code>sublinear_tf</code>	<code>True</code> ($1 + \log(\text{tf})$)
<code>strip_accents</code>	<code>"unicode"</code>

3.2 Sieć gęsta (MLP)

Klasyfikatorem jest wielowarstwowa sieć gęsta (`MLPClassifier`, `src/model/mlp.py`) o architekturze:

$$\underbrace{\text{Linear}(50\,000, 256)}_{\text{warstwa ukryta}} \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0,3) \rightarrow \underbrace{\text{Linear}(256, 1)}_{\text{wyjście (logit)}}.$$

Wyjściem jest pojedynczy *logit* (bez sigmoidy), co pozwala użyć numerycznie stabilnej funkcji straty `BCEWithLogitsLoss`. Sieć liczy **12 800 513** parametrów, z czego zdecydowana większość przypada na pierwszą warstwę ($50\,000 \times 256$), bo wymiar wejścia równy jest rozmiarowi słownika.

3.3 Proces uczenia

Funkcja `train_mlp` realizuje pełną pętlę treningową z **early stopping** monitorującym F1 na zbiorze walidacyjnym i przywracającym wagi z najlepszej epoki (Tabela 4). W praktyce trening zatrzymał się po **11 epokach** (z dozwolonych 20).

Tabela 4: Hiperparametry uczenia.

Parametr	Wartość
optymalizator	Adam
learning rate	1×10^{-3}
funkcja straty	BCEWithLogitsLoss
rozmiar batcha	256
maks. epok	20
early stopping	na F1 walidacji, <code>patience</code> = 3
dropout	0,3
weight decay (L2)	0 (domyślnie; badany w sekcji 4)
próg decyzyjny	0,5
ziarno losowe	RANDOM_STATE (powtarzalność)

Uzasadnienie wyborów. Funkcja `BCEWithLogitsLoss` łączy sigmoidę z entropią krzyżową w sposób numerycznie stabilny i jest naturalna dla klasyfikacji binarnej. Adam zapewnia szybką zbieżność bez ręcznego strojenia harmonogramu uczenia. Early stopping na F1 (a nie na stracie) jest spójny z metryką raportowaną i chroni przed przeuczeniem przy praktycznie zerowym koszcie. Dropout 0,3 to standardowa, umiarkowana regularyzacja warstwy ukrytej.

4 Ewaluacja

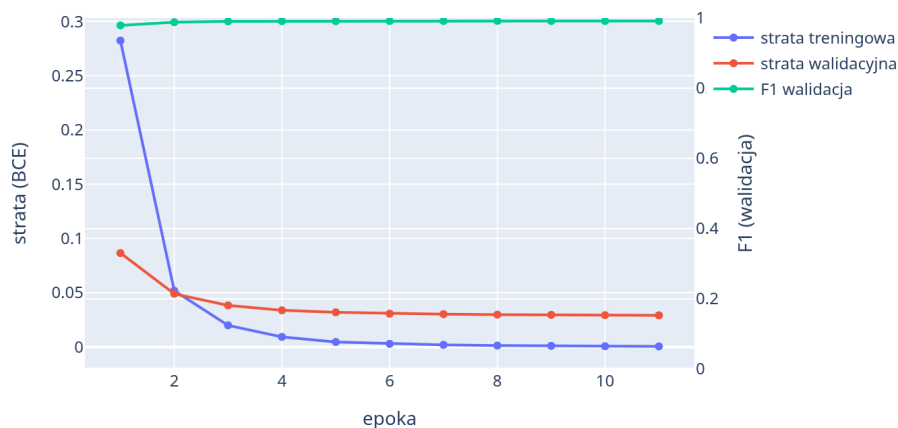
4.1 Metryki na walidacji i teście

Model oceniono pięcioma metrykami (Tabela 5). Wszystkie utrzymują się na poziomie $\sim 0,99$, co dla tego zbioru oznacza praktyczny sufit jakości.

Tabela 5: Metryki MLP na zbiorze walidacyjnym i testowym.

Metryka	Walidacja	Test
accuracy	0,9908	0,9864
precision	0,9899	0,9838
recall	0,9934	0,9915
F1	0,9916	0,9877
ROC-AUC	0,9992	0,9987

Krzywe uczenia MLP

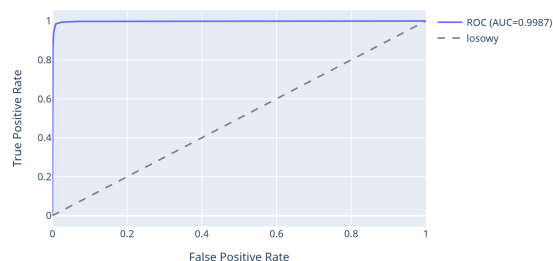


Rysunek 1: Krzywe uczenia MLP: strata treningowa/walidacyjna (oś lewa) oraz F1 walidacji (oś prawa). Sieć zbiega w pierwszych epokach; early stopping przerywa trening, gdy F1 przestaje rosnąć.

Macierz pomyłek — zbiór testowy (MLP)

Prawdziwa klasa	Fake (0)	3387	69
	Real (1)	36	4202
		Fake (0)	Real (1)
		Predykcja	

Krzywa ROC — zbiór testowy (MLP)



Rysunek 2: Macierz pomyłek (po lewej) i krzywa ROC (po prawej) na zbiorze testowym. Błędy są nieliczne i rozłożone na obie klasy.

4.2 Porównanie z baselinem (LogReg vs MLP)

Ponieważ oba modele korzystają z *identycznej* reprezentacji TF-IDF, różnica wyników izoluje wpływ samej architektury klasyfikatora (Tabela 6).

Tabela 6: Porównanie na zbiorze testowym.

Metryka	LogReg (baseline)	MLP (od zera)
accuracy	0,9795	0,9864
precision	0,9751	0,9838
recall	0,9880	0,9915
F1	0,9815	0,9877
ROC-AUC	0,9970	0,9987

MLP przewyższa baseline o ok. 0,6 pp F1 (różnica marginalna). Na tym leksykalnie łatwym zbiorze klasy są niemal liniowo separowalne w przestrzeni TF-IDF, więc nieliniowość sieci nie

daje istotnej przewagi. Wartością tego modelu jest nie sam wzrost metryk, lecz kompletna, zweryfikowana pętla treningowa w PyTorchu oraz rzetelna eliminacja wycieku danych.

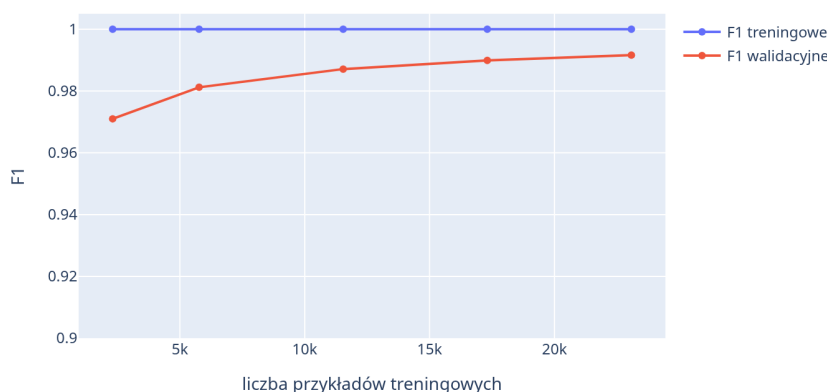
4.3 Diagnoza bias-variance

Krzywa uczenia (trening na rosnących podzbiorach 10–100% przy ustalonym wektoryzatorze) izoluje wpływ liczby przykładów. Obie krzywe F1 leżą wysoko i blisko siebie (Tabela 7, Rys. 3): F1 treningowe $\approx 1,0$ (**niski bias**), a luka train-val maleje do $\sim 0,8$ pp wraz z liczbą przykładów (**niska wariancja**). Krzywa walidacyjna szybko wchodzi na plateau, co sugeruje, że dokładanie danych przyniosłoby co najwyżej marginalną poprawę na tym zbiorze.

Tabela 7: Krzywa uczenia: F1 vs liczba przykładów treningowych.

n_{train}	F1 train	F1 val	luka
2 308	1,0000	0,9710	0,0290
5 770	1,0000	0,9812	0,0188
11 540	1,0000	0,9871	0,0129
17 312	1,0000	0,9899	0,0101
23 082	1,0000	0,9916	0,0084

Krzywa uczenia — diagnoza bias/variance



Rysunek 3: Krzywa uczenia. Obie krzywe wysoko i blisko siebie: reżim niskiego biasu i niskiej wariancji przy suficie metryk zbioru.

4.4 Techniki regularyzacji

Choć przeuczenie jest tu łagodne, model umożliwia jego kontrolę trzema technikami: early stopping (zawsze aktywny), dropout i weight decay (L2). Tabela 8 pokazuje wpływ na lukę train-val (miarę wariancji).

Tabela 8: Wpływ regularyzacji na lukę train-val F1.

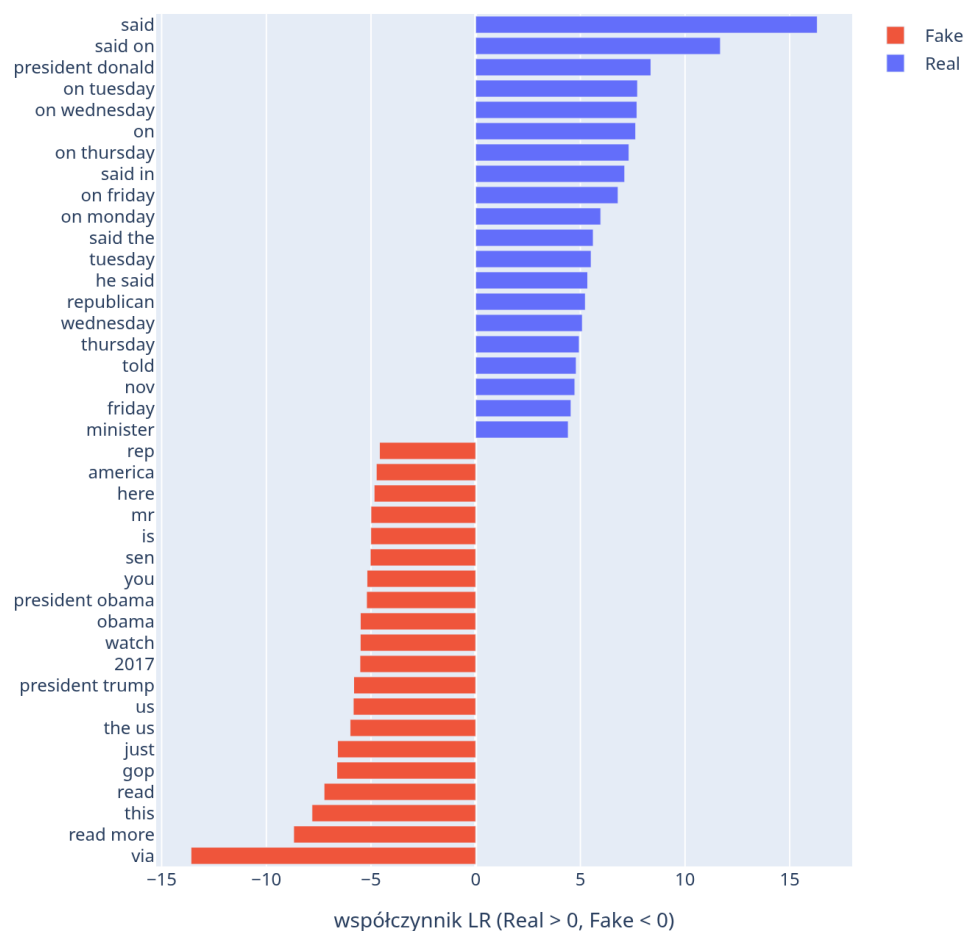
Konfiguracja	F1 train	F1 val	luka
brak (dropout=0, wd=0)	1,0000	0,9913	0,0087
dropout=0,3	1,0000	0,9916	0,0084
L2 (wd= 10^{-4})	0,9987	0,9939	0,0048
dropout=0,3 + L2	0,9987	0,9933	0,0054

Dropout i weight decay zmniejszają lukę (obniżają F1 treningowe, podtrzymując walidacyjne), lecz F1 walidacyjne ledwie drga; przy tak łatwym zbiorze regularyzacja głównie kosmetyzuje lukę, a nie poprawia uogólnienia. Potwierdza to diagnozę z krzywej uczenia.

4.5 Interpretowalność cech

Współczynniki regresji logistycznej (baseline) dają tanią, globalną wyjaśnialność (Rys. 4). Stronę Real ciągną zwroty agencyjno-sprawozdawcze (`said`, `on wednesday`, `president donald`), a stronę Fake zwroty potoczne/clickbaitowe (`via`, `read more`, `you`, `watch`, `video`). Co istotne, *żaden* z usuniętych markerów wycieku nie pojawia się na liście, więc model uczy się stylu i tematu, a nie stempla redakcyjnego.

Najsilniejsze n-gramy wg współczynników regresji logistycznej



Rysunek 4: Najsilniejsze n-gramy wg współczynników regresji logistycznej (Real > 0, Fake < 0).

5 Przykładowe predykcje

Aby pokazać działanie sieci na konkretnych artykułach, dla każdego tekstu ze zbioru testowego obliczono prawdopodobieństwo $P(\text{Real})$ i porównano z progiem 0,5. Tabela 9 zestawia dwa pewne trafienia każdej klasy oraz oba typy błędów: *false positive* (Fake uznane za Real) i *false negative* (Real uznane za Fake).

Tabela 9: Przykładowe predykcje MLP na zbiorze testowym (próg 0,5).

Fragment tekstu	Prawdziwa	P(Real)	Predykcja	
<i>Pewne trafienia</i>				
„germany foreign minister said on saturday that...”	Real	1,0000	Real	✓
„china called for ceasefire in myanmar rakhine...”	Real	1,0000	Real	✓
„patrick henningsen the longer this soap opera...”	Fake	0,0000	Fake	✓
„patrick henningsen despite repeated failures...”	Fake	0,0000	Fake	✓
<i>Błędy: false positive (Fake → Real)</i>				
„the washington post nearly third of territory...”	Fake	0,9998	Real	×
„leading from behind hasn worked out so well...”	Fake	0,9997	Real	×
<i>Błędy: false negative (Real → Fake)</i>				
„- below are the highlights from oct. 25 excl...”	Real	0,0015	Fake	×
„the emmy awards show was peppered with politic...”	Real	0,0059	Fake	×

Kalibracja i omówienie błędów. Prawdopodobieństwa $P(\text{Real})$ są **nasycone na krańcach** (≈ 0 dla Fake, ≈ 1 dla Real), również w przypadku pomyłek: artykuły Fake błędnie zaklasyfikowane jako Real otrzymują $P(\text{Real}) \approx 0,9998$. Model jest zatem **silny w rankingu** (ROC-AUC 0,9987), lecz **przepewny i słabo skalibrowany**: myli się rzadko, ale gdy się myli, robi to z wysoką pewnością, zamiast sygnalizować niepewność wartościami w pobliżu progu 0,5. Same błędy dotyczą tekstów stylistycznie „pogranicznych”, co *sugeruje* (lecz nie dowodzi) związek z cechami z sekcji 4:

- **False positive** (Fake → Real): teksty otwierające się frazą agencyjno-prasową (np. „the washington post...”), stylistycznie zbliżone do źródeł wiarygodnych;
- **False negative** (Real → Fake): teksty nietypowe dla klasy Real, sformatowane jako lista („- below are the highlights...”) lub o tematyce rozrywkowej (gala Emmy) zamiast politycznej/agencyjnej.

Błędy są więc rzadkie i koncentrują się na artykułach „pogranicznych” stylistycznie, co jest spójne z nielicznymi pomyłkami w macierzy pomyłek (Rys. 4). Wysoka pewność tych pomyłek pozostaje jednak sygnałem ograniczonej kalibracji modelu.

6 Kluczowe komponenty kodu

Cała logika modelu jest wydzielona do `src/model/mlp.py` i pokryta testami jednostkowymi (`tests/model/`). Poniżej omówiono najistotniejsze fragmenty.

6.1 TfidfDataset: leniwa densyfikacja

Macierz TF-IDF jest rzadka (50 000 kolumn, lecz nieliczne niezerowe wartości w wierszu). Gęste zmaterializowanie całego korpusu naraz byłoby kosztowne pamięciowo, dlatego `TfidfDataset` przechowuje dane jako macierz rzadką i densyfikuje *pojedynczy wiersz dopiero przy pobraniu* przez `DataLoader`.

```

1 def __getitem__(self, index: int) -> tuple[torch.Tensor, torch.Tensor]:
2     row = self.features[index].toarray().ravel().astype(np.float32, copy
3         =False)
4     return torch.from_numpy(row), self.labels[index]

```

Listing 1: Leniwa densyfikacja wiersza w TfidfDataset.

6.2 MLPClassifier: sieć zwracająca logity

Sieć budowana jest jako stos bloków `Linear` → `ReLU` → `Dropout`, zakończony pojedynczą warstwą liniową. `forward` zwraca surowe *logity* (bez sigmoidy), co pozwala użyć numerycznie stabilnej `BCEWithLogitsLoss` w treningu, a `torch.sigmoid` dopiero przy wnioskowaniu.

```

1 layers: list[nn.Module] = []
2 in_features = input_dim
3 for width in hidden_dims:
4     layers.append(nn.Linear(in_features, width))
5     layers.append(nn.ReLU())
6     layers.append(nn.Dropout(dropout))
7     in_features = width
8 layers.append(nn.Linear(in_features, 1))
9 self.net = nn.Sequential(*layers)

```

Listing 2: Konstrukcja warstw MLPClassifier.

6.3 train_mlp: pętla treningowa z early stopping

Pętla naprzemiennie trenuje (`_train_one_epoch`) i waliduje (`_validate`, zwraca stratę i F1). Po każdej epoce, jeśli F1 walidacji poprawił się, zapisywany jest stan wag; w przeciwnym razie zliczane są epoki bez poprawy, a po przekroczeniu *patience* trening zostaje przerwany. Na końcu przywracane są najlepsze wagi.

```

1 optimizer = torch.optim.Adam(model.parameters(), lr=lr, weight_decay=
2     weight_decay)
3 criterion = nn.BCEWithLogitsLoss()
4 ...
5 for _ in range(epochs):
6     train_loss = _train_one_epoch(model, train_loader, optimizer,
7         criterion, dev)
8     val_loss, val_f1 = _validate(model, val_loader, criterion, dev)
9     ...
10    if val_f1 > best_val_f1:
11        best_val_f1 = val_f1
12        best_state = {n: t.detach().cpu().clone()
13            for n, t in model.state_dict().items()}
14        epochs_without_improvement = 0
15    else:
16        epochs_without_improvement += 1
17        if epochs_without_improvement >= patience:
18            break
19 if best_state:
20     model.load_state_dict(best_state)

```

Listing 3: Rdzeń early stopping w train_mlp.

6.4 predict_proba i evaluate_mlp

`predict_proba` przepuszcza dane przez sieć w trybie ewaluacji i zwraca $P(\text{Real})$ jako tablicę `numpy` (`logit` $\rightarrow$ `sigmoid`). `evaluate_mlp` owija to w obliczenie metryk i zwraca *dokładnie taki sam słownik* (`accuracy`, `precision`, `recall`, `f1`, `roc_auc`, `confusion_matrix`) co `baseline.evaluate`, dzięki czemu porównanie obu modeli jest bezpośrednie.

6.5 Reprodukowalność i urządzenie

`set_seed` ustawia ziarno dla NumPy oraz PyTorch (CPU i CUDA/ROCm), a `resolve_device` automatycznie wybiera GPU, gdy jest dostępne (na maszynie deweloperskiej działa `build ROCm`, dla którego `torch.cuda.is_available()` zwraca `True`). Oba modele korzystają z tego samego wektoryzatora (`baseline.make_tfidf_vectorizer`), co gwarantuje identyczną reprezentację tekstu i uczciwe porównanie architektur.

7 Wnioski i dalsze kroki

7.1 Podsumowanie

- **TF-IDF** $\rightarrow$ **MLP** to sieć neuronowa zbudowana w całości od zera (bez *fine-/feature-tuning*), zgodna z wymogiem przedmiotu SSN. Działa na tych samych splitach, konwencji etykiet i wektoryzatorze co `baseline`, a cała logika jest reużywalna w `src/model/mlp.py`.
- Model osiąga F1 i ROC-AUC na poziomie $\sim 0,99$ na zbiorze testowym, co jest wynikiem porównywalnym z `baseline` (regresją logistyczną). Na tym leksykalnie łatwym zbiorze sieć nie zyskuje istotnej przewagi nad modelem liniowym.
- Diagnoza **bias-variance** (krzywa uczenia + sweep regularyzacji) wskazuje reżim **niskiego biasu i niskiej wariancji** przy suficie metryk zbioru: powiększanie modelu ani zbioru przyniosłoby co najwyżej marginalną poprawę, a regularyzacja jedynie domyka niewielką lukę `train-val`.
- Przykładowe predykcje ujawniają, że model jest **przepewny**: prawdopodobieństwa są nasycone na krańcach również przy pomyłkach (słaba kalibracja mimo wysokiego ROC-AUC). Nieliczne błędy dotyczą tekstów stylistycznie „pogranicznych” (agencyjny styl w artykułach Fake, format listy lub tematyka rozrywkowa w artykułach Real).

7.2 Ograniczenia

Najważniejszym ograniczeniem jest **sufit metryczny** zbioru: po usunięciu wycieku klasy nadal różnią się na tyle silnie leksykalnie, że proste reprezentacje (TF-IDF) wystarczają do niemal perfekcyjnej klasyfikacji. Utrudnia to wykazanie przewagi bardziej złożonych architektur na tym konkretnym zbiorze. Model worka słów (TF-IDF) z założenia ignoruje też **kolejność i kontekst** słów.

7.3 Dalsze kroki

Projekt dostarcza zweryfikowaną pętlę treningową PyTorch (Adam, `BCEWithLogitsLoss`, early stopping, dropout, weight decay, obsługa GPU) oraz kompletny, reużywalny kod modelu w `src/model/mlp.py`. Naturalnym kierunkiem dalszego rozwoju, poza zakresem niniejszego projektu, jest architektura **BiLSTM + Attention** z osadzeniami **GloVe 300d**: w odróżnieniu od worka słów modelowałaby kontekst sekwencyjny, a mechanizm atencji umożliwiłby **wyjaśnialność na poziomie pojedynczego artykułu** (wizualizację wag atencji).

Literatura

- [1] C. Bisailon, *Fake and Real News Dataset*, Kaggle. <https://www.kaggle.com/datasets/clmentbisailon/fake-and-real-news-dataset>.
- [2] J. Pennington, R. Socher, C. D. Manning, *GloVe: Global Vectors for Word Representation*, EMNLP 2014.
- [3] A. Paszke i in., *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, NeurIPS 2019.
- [4] F. Pedregosa i in., *Scikit-learn: Machine Learning in Python*, JMLR 12, 2011.